Linear-Cost Scaling: What Grows Linearly in CKS, What Doesn't, and Why

Author: Wenxin Li (Independent Researcher)
ORCID: 0009-0004-8065-3235
Date: 25 April 2026
License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Attribution

This work derives from and formalizes the Coordination Knowledge Substrate (CKS) pattern introduced in *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems* (Li, April 2026). It does not introduce new axioms. Its sole contribution is to articulate, in operational form, the cost commitment the CKS pattern makes — what scales with what, and what does not — so that downstream work can adopt or argue against the commitment without ambiguity, and so that implementers have a single reference for what their system must preserve to remain CKS-coherent on the cost axis.

Abstract

The CKS pattern names "linear-cost in storage and composition" as one of its six architectural commitments, defended in the source paper as Claim 4 (§6). The commitment is widely misread as a claim about storage alone — the trivial property that any persistent representation has storage cost proportional to content volume. The substantive claim is the asymmetric one: of the costs a CKS system incurs, *only* storage and full-substrate read scale with substrate size; governance cost, cell execution cost, LLM cost per execution, conflict-handling cost, and onboarding cost for new readers do not. This note formalizes the cost contract along three independent dimensions (substrate size, rule variety, intervention frequency), separates the costs that scale with each, distinguishes the CKS profile from three adjacent profiles (parametric memory, external structured memory in the Knowledge Objects family, and workflow-engine state), and shows that the linear-cost property is an emergent consequence of the other commitments rather than an independent axiom. An operational test follows.

1. Why the cost profile needs to be stated as a single contract

The CKS pattern uses "linear-cost in storage and composition" as the architectural anchor for its claim of bottom-up adoptability — that a small team can begin with a single cell, grow the substrate incrementally, and not encounter an infrastructure cliff at any point. That claim is only defensible if the cost curve actually stays linear in substrate size, and the source paper develops the case across §6 in three pieces: §6.1 distinguishes the three sub-claims; §6.2 inherits the database-like cost curve from Knowledge Objects (Zahn & Chana 2026) and contrasts it with the superlinear curves of parametric memory and agent coordination; §6.3 separates the labor curve of authoring substrate content from the infrastructure curve the linear-cost claim is about.

Distributed across three subsections, the cost commitment is harder to cite as a single contract than its load-bearing role warrants. This note assembles it into one statement, organized around a three-dimensional

cost decomposition: the linear-cost commitment is a statement about what scales with substrate size and what does not, while the other two dimensions carry the costs the pattern does pay. The contribution is articulation, not extension; every claim is traceable to the source paper or to the prior derivation note on "human-governed" (Li, 24 April 2026).

2. Three cost dimensions

A CKS system has three independent dimensions along which cost may be incurred. The linear-cost commitment is a statement about the dependency structure between each cost in the system and these three dimensions.

Dimension A — Substrate size. The total volume of substrate content the system carries: entities, relationships, decisions, rationale, and preserved conflicts. Substrate size grows as the system accumulates coordination knowledge over time.

Dimension B — Rule variety. The number and complexity of orchestration rules governing cell behavior. Rule variety grows when the system needs to support new cell behaviors — a new conflict-handling pattern, a new cell-level workflow, a new authority partition between roles.

Dimension C — Intervention frequency. The rate at which humans exercise their preserved override right to directly modify substrate content or orchestration rules. Intervention frequency is bounded by operator choices and stakes, not by content volume.

A fourth quantity — cell execution count, the number of times cells run over substrate content — is a workload property that varies with task volume independently of all three dimensions, and the linear-cost commitment is not a statement about it.

3. What scales with which dimension

Naming the costs that *do* grow with each dimension is the first step toward making explicit what the linear-cost commitment denies in §4.

Costs that scale with Dimension A (substrate size).

(a) *Storage cost.* Substrate content occupies storage proportional to its volume. This is a property of any persistent representation and is not distinctive to CKS.

(b) *Read cost for full-substrate operations.* When a cell or a reader requires every element of substrate content, that read is proportional to substrate size. Well-designed cells rarely require this; most operate over a task-scoped subset.

(c) *Per-element inspection cost.* Exercising the inspect right on every element of substrate content is, by construction, proportional to substrate size. The architecture preserves the *option* of paying this cost; it does not require it. The distinction matters in §4.

Cost that scales with Dimension B (rule variety).

(d) *Orchestration rule authoring cost.* Each new rule requires authoring effort once, paid at design time, amortizing across every subsequent cell execution under the rule. Rule authoring cost is independent of how much substrate content exists or will exist.

Cost that scales with Dimension C (intervention frequency).

(e) *Override cost.* Each human-initiated direct change to substrate content or orchestration rules costs the human's time at the moment of change. Override cost is paid only when an override is exercised.

Cost that scales with cell execution count.

(f) *Per-execution cost.* Running a cell incurs the cost of the cell's read, the LLM operations the cell performs, and the cell's writes. Per-execution cost grows with the number of executions, not with substrate size.

The linear-cost commitment fixes how each of these depends on Dimension A, and the load-bearing claim is in §4.

4. What does NOT scale with substrate size

The substantive content of the linear-cost commitment is the asymmetry between what depends on Dimension A and what does not. Five costs that might naively be expected to grow with substrate size, do not.

Governance cost does not scale with substrate size. Governance is exercised at the two moments the prior derivation note identifies: orchestration rule authoring (Dimension B) and direct override (Dimension C). Neither is per-element review. A substrate that grows from 100 entries to 100,000 entries does not require 1,000× more governance work, because governance is not a function the architecture asks humans to apply per element. This is the property the source paper renders portable as *governance is an authority architecture, not a review workflow* (§3.3).

Cell execution cost does not scale with substrate size. A well-designed cell reads only the substrate content within its task scope, not the full substrate. Cell execution cost depends on the cell's task scope and the size of the content the cell touches, not on the total volume the system holds. Adding cells in unrelated regions of the substrate does not slow down a cell whose scope does not extend into those regions.

LLM cost per cell execution does not scale with substrate size. The LLM operates over the substrate content the cell reads, not over the full substrate. This depends on the AI-as-substrate-mediator commitment: the LLM does not hold substrate-relevant state across executions, so each execution's LLM cost is a function of what it reads and writes, not of substrate history. A substrate that grows tenfold does not require the LLM to attend over tenfold more content per execution.

Conflict-handling cost does not scale with substrate size. Conflicts are preserved as first-class addressable substrate content and resolved at cell-execution time under orchestration rules, not by global reconciliation. Adding new substrate content does not trigger a re-reconciliation pass over existing content. The cost of handling a conflict is local to the cell that encounters it; it is paid once per encounter, not every time the substrate grows.

Onboarding cost for new readers does not scale with substrate size. A reader exercising the inspect right reads what they need, not the full substrate. The cost of onboarding a new participant — a new reviewer, a new auditor, a new role — depends on what that participant needs to read for their role, not on the total volume of substrate content. This is the architectural feature that makes non-specialist governance per §7.4 meaningful: the inspect right is exercisable without first reading every element of the substrate.

The asymmetry these five points describe is the substantive linear-cost claim. Storage growing linearly with content (§3(a)) is trivial; the claim that *nothing else* grows with substrate size is what makes the cost profile distinctive.

5. Contrast with adjacent cost profiles

Three adjacent cost profiles deserve naming to make the CKS commitment crisp.

Parametric memory. Encoding new knowledge into model weights — fine-tuning, continual learning, knowledge editing — incurs training cost that scales superlinearly with the volume of new knowledge, with retraining cost scaling as the entire model rather than as the increment. The 2024–2026 evidence the source paper cites (Wang et al.; the mechanistic study of catastrophic forgetting at the parameter level; ROME, MEMIT, and MEND scaling-ceiling results) shows the curve bending at small edit counts. The CKS cost model is structurally different: new substrate content has no training cost, no parameter-update cost, and no retraining cost. Knowledge accretion in CKS is a write operation on persistent storage, not a modification of model state.

External structured memory (Knowledge Objects, OIDA-family). The closest neighbor. Adding new entries to an external structured store scales linearly in storage and approximately constant in per-query token cost, as Knowledge Objects (Zahn & Chana 2026) demonstrates with O(1) retrieval that holds across three orders of magnitude of corpus growth. CKS inherits this base curve. The distinction the source paper makes (§6.2) is on the multi-human axis: governance cost in an OIDA-style architecture scales with the number of human reviewers required to maintain content, while CKS — by allocating governance to the two non-size-proportional moments named in §4 above — keeps governance cost bounded by rule variety and intervention frequency rather than by reviewer count. The cost profiles agree on storage and lookup; they differ on the governance-cost axis.

Workflow-engine state. Workflow engines that maintain persistent shared state across many concurrent processes can incur coordination overhead that grows superlinearly with state volume, through lock contention, transaction conflicts, or global reconciliation passes. CKS avoids this by handling conflicts at cell-execution time under orchestration rules, not at global state-update time. There is no global reconciliation pass in CKS; preserved conflicts are addressable substrate content, not a queue the system must drain.

6. Why this profile depends on the other commitments

The linear-cost property is not an independent axiom. It is an emergent consequence of the other commitments, and stating the dependency arrows explicitly is what makes the cost profile derivable rather than asserted.

Human-governed → governance cost is independent of substrate size. The human-governed commitment names authority, not labor (per the prior derivation note). Authority is exercised at orchestration rule authoring and at direct override, neither of which is per-element. Read instead as human-reviewed-everything, governance cost would scale with Dimension A and the linear-cost commitment would fail. The authority-vs-labor distinction is the load-bearing dependency.

AI-as-substrate-mediator → LLM cost per execution is independent of substrate size. Because the LLM mediates the substrate rather than carrying substrate state in its weights or in-context memory, each

cell execution's LLM cost is a function of what the execution reads, not of substrate history. An LLM that held substrate-relevant state across executions — fine-tuned on substrate content, or relying on a continually expanding context window of accumulated substrate state — would make LLM cost per execution scale with substrate size, and the linear-cost commitment would fail.

Conflict preservation with cell-level resolution → conflict cost is local, not global. Conflicts are preserved as first-class addressable substrate content, and the source paper's two-level conflict-handling design (§5.3) places resolution at cell-execution time under orchestration rules. An architecture that required global reconciliation — a pass over all substrate content whenever new content is added — would make conflict-handling cost scale with substrate size.

Tool-agnosticism → no specialized runtime middleware introduces size-proportional overhead. The substrate-layer tool-agnosticism the source paper defends in §7 means the pattern instantiates in environments meeting three minimal requirements (persistent structured state, human read/write access, LLM access to substrate content), without a specialized governance runtime sitting between the substrate and its users. A required runtime maintaining per-element governance state would make governance overhead scale with Dimension A.

The four implication arrows show that linear-cost scaling is not an independent claim defended on its own evidence; it is what the architecture yields when the other commitments are preserved. The point is reinforced by §11.3's framing of the substrate as the source of truth: growth happens in the substrate itself, not in model state or runtime middleware, which is what makes Dimension A the meaningful axis along which the system grows. A system that preserves human-governed, AI-as-substrate-mediator, conflict preservation with cell-level resolution, and tool-agnosticism preserves the cost profile by construction; a system that violates any of these dependencies will exhibit a cost profile that fails the linear-cost commitment, regardless of intent.

7. Operational test, and what the commitment rules out

A system preserves the CKS linear-cost commitment if and only if all of the following are true at all times during the substrate's existence:

1. Adding substrate content does not increase governance cost.
2. Adding substrate content does not increase cell execution cost for cells whose task scope does not require the new content.
3. Adding substrate content does not require LLM retraining, fine-tuning, or any operation whose cost scales with substrate size.
4. Adding substrate content does not require global reconciliation, reprocessing, or revalidation of existing substrate content.
5. The system permits — but does not require — per-element inspection of substrate content. Making inspection mandatory would convert the optional cost in §3(c) into a required size-proportional cost and would violate the commitment, even though the architecture preserves the inspect right itself.

Implementations that fail any of (1)–(5) are not CKS-coherent on the cost axis. They may be useful for other purposes; they may be governed in some other sense; they may inherit from CKS at the schema or substrate level. But on the cost axis they have moved into a different architectural region. Concrete violations the test

surfaces include: governance procedures that require per-element review at a scheduled cadence (violates 1), cells whose default behavior is to read the full substrate rather than a task-scoped subset (violates 2), continual-learning loops that fine-tune on accumulated substrate content (violates 3), and consistency-maintenance passes triggered by writes (violates 4).

Naming the cost profile explicitly is what makes the CKS pattern's bottom-up adoptability defensible as an architectural property rather than a marketing claim. The pattern instantiates in commodity infrastructure (per Claim 5) and grows incrementally without infrastructure cliffs (per Claim 4) precisely because the cost profile is asymmetric in the way §4 describes. Subsequent work that adopts the CKS pattern, extends it, composes it with adjacent patterns, or argues against it should use "linear-cost scaling" in the sense formalized here. Subsequent work that uses the term differently — most often by identifying the commitment with storage alone, while leaving size-proportional governance, execution, or model-state operations unconstrained — is using a different concept, and the difference should be named.

Source paper

Li, W. (2026). *Coordination Outside the Model: A Human-Governed Substrate Pattern for AI Systems.* April 2026. ORCID: 0009-0004-8065-3235.

Companion derivation note

Li, W. (2026). *Authority, Not Labor: A Precise Definition of "Human-Governed" in the Coordination Knowledge Substrate Pattern.* 24 April 2026. ORCID: 0009-0004-8065-3235.

How to cite this note

Li, W. (2026). *Linear-Cost Scaling: What Grows Linearly in CKS, What Doesn't, and Why.* 25 April 2026. ORCID: 0009-0004-8065-3235.